
NEORV32 Ascon-AEAD128 Hardware Accelerator

Project Report

Results, methodology, and engineering challenges of an
open-source Ascon accelerator validated on silicon.

Headline result

All 8 test cases correct on hardware; the accelerator is
 $\sim 18\times$ to $\sim 57\times$ faster than the software reference
on the same RISC-V core.

Mohammadi Masoudi Ramin

Politecnico di Milano

Contents

1	Executive Summary	2
2	Objectives and Method	2
2.1	Objectives	2
2.2	Method	2
3	The Test Platform	3
4	Results	3
4.1	Correctness	3
4.2	Performance	3
4.3	Cycle counts, visualised	4
4.4	Speed-up trend	4
5	What the Speed-up Is Measured Against	4
6	Engineering Challenges	5
6.1	The pattern that mattered	6
7	Design Choices and Their Consequences	6
8	Limitations and Future Work	6
9	Conclusion	7

1 Executive Summary

This project designed, verified, and validated on silicon a hardware accelerator for Ascon-AEAD128, the NIST lightweight authenticated-encryption standard (SP 800-232). The accelerator was generated from a typed Python golden model, integrated as an external-bus peripheral on a fully-open NEORV32 RISC-V system-on-chip, built end-to-end with an all-open toolchain (GHDL, Yosys, nextpnr, Apicula), and programmed onto a Tang Nano 20K FPGA board.

On hardware, an on-chip benchmark encrypted and decrypted eight associated-data/plaintext combinations and checked every result against a software reference running on the same CPU. **All eight cases were correct**, and the hardware was between **17.6× and 57.4× faster** than the reference software, depending on payload size. The remainder of this report explains what was measured, how, the engineering challenges overcome, the design choices behind them, and an honest account of what the numbers do and do not claim.

2 Objectives and Method

2.1 Objectives

1. Implement Ascon-AEAD128 in hardware, provably matching the standard.
2. Make it usable from software through a clean C interface on a RISC-V CPU.
3. Build the whole system with open-source tools only, reproducibly.
4. Validate on real silicon and quantify the speed-up over software.

2.2 Method

The accelerator was generated from a golden model that passes the NIST Known-Answer Tests, then verified in four layers (model → RTL → driver → end-to-end co-simulation; 119 automated tests). It was integrated on NEORV32, synthesised with the open flow, and flashed to the board. On-chip, a benchmark timed a software Ascon reference and the hardware accelerator on the same inputs, verified equality, and reported cycle counts. The architecture and verification are detailed in the companion Design Document; this report concentrates on the outcome.

3 The Test Platform

Component	Configuration
FPGA board	Sipeed Tang Nano 20K
FPGA device	Gowin GW2AR-18 (GW2A-18C family)
Soft CPU	NEORV32, rv32i + Zicntr, no M extension
Clock	27 MHz (board oscillator; timing closed with $\sim 2\times$ margin)
Memory	IMEM 16 KB, DMEM 8 KB (on-chip block RAM)
Accelerator bus	Wishbone-b4 on NEORV32 XBUS, base 0x9000 0000
Toolchain	GHDL $\rightarrow$ Yosys $\rightarrow$ nextpnr-himbaechel $\rightarrow$ Apicula (all open)
Firmware toolchain	RISC-V GCC, freestanding, soft-float

4 Results

4.1 Correctness

Across all eight cases, hardware encryption produced ciphertext and tag identical to the software reference; hardware decryption recovered the plaintext with a valid tag; and (in the standalone self-test) a tampered ciphertext was correctly rejected. Every `enc_ok`, `dec_ok`, and `tag_valid` flag was set. The on-chip verdict was **PASS** on all eight of eight cases.

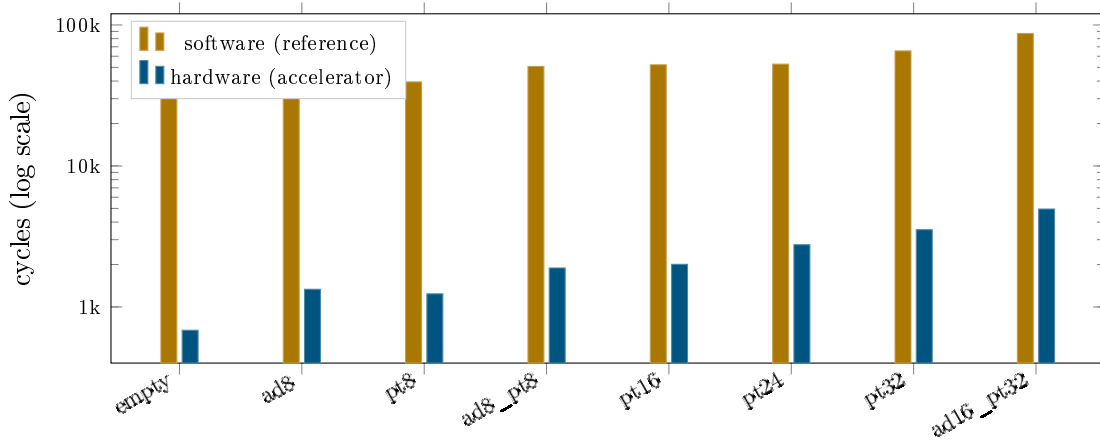
4.2 Performance

The measured cycle counts and resulting speed-ups (from the on-chip benchmark):

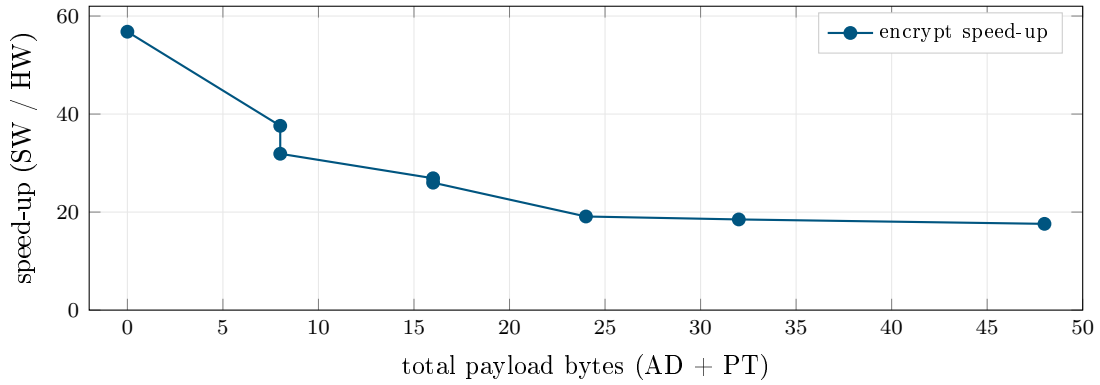
Case	AD	PT	SW enc	HW enc	Enc $\times$	SW dec	HW dec	Dec $\times$	OK
empty	0	0	38 866	684	56.8	39 241	684	57.4	✓
ad8	8	0	50 205	1 335	37.6	50 579	1 335	37.9	✓
pt8	0	8	39 458	1 239	31.9	39 665	1 239	32.0	✓
ad8_pt8	8	8	50 797	1 890	26.9	51 003	1 890	27.0	✓
pt16	0	16	52 156	2 007	26.0	52 595	2 007	26.2	✓
pt24	0	24	52 748	2 763	19.1	53 019	2 763	19.2	✓
pt32	0	32	65 447	3 531	18.5	65 950	3 531	18.7	✓
ad16_pt32	16	32	86 984	4 948	17.6	87 486	4 948	17.7	✓

AD and PT are associated-data and plaintext bytes. SW/HW columns are clock cycles at 27 MHz. Speed-up is $\text{SW} \div \text{HW}$.

4.3 Cycle counts, visualised



4.4 Speed-up trend



The speed-up is largest for tiny payloads and falls as payloads grow. This is the expected shape and is explained by *where the hardware spends its cycles*: the hardware figure includes the time the CPU takes to move data in and out of the accelerator word-by-word over the memory-mapped interface, and that data-movement cost grows with payload size while the fixed cryptographic cost does not.

5 What the Speed-up Is Measured Against

Being precise here matters, because “57× faster” is meaningless without saying faster *than what*.

- **Software (numerator):** CPU clock cycles, read with `rdcycle`, around a clean 64-bit C reference implementation of Ascon-AEAD128 (the straightforward NIST-style code), running on the same NEORV32 core (a 32-bit `rv32i` CPU at 27MHz). Because the core is 32-bit, each 64-bit Ascon word costs several instructions.
- **Hardware (denominator):** the accelerator’s own cycle-counter register, i.e. the number of clock cycles it was busy for the operation, at the same 27MHz.

So the honest phrasing is: *the hardware crypto engine versus a reference software Ascon on the same SoC*. It is a fair, same-platform, same-clock comparison.

Honest caveat: The software baseline is a *reference*, not a hand-optimised or assembly implementation — an optimised software Ascon on the same core would be faster, narrowing the gap (though hardware would still win). And the hardware counter measures the operation from start to done, so it *excludes* the register-setup writes before and the tag/output reads after. A fully end-to-end wall-clock comparison (CPU cycles around the entire driver call) would show somewhat lower speed-ups, most noticeably for the tiny `empty` case. The $\sim 18\times$ on real payloads is closer to the end-to-end story than the $\sim 57\times$ on the empty one.

6 Engineering Challenges

The path from “simulation passes” to “correct on silicon” surfaced a sequence of real bugs. Each was diagnosed from concrete evidence — a linker error, a processor panic with a fault address, a serial trace — rather than guesswork. They are worth recording, because they are exactly the class of problem a simulation-only flow does not reveal.

1 — Read-only build environment

The NEORV32 build compiled a host helper tool and memory images *into* its own source tree, which on this setup lives in the read-only Nix store. Fixed by redirecting all generated artefacts into a writable build directory while still reading sources from the store.

2 — Toolchain name mismatch

The firmware Makefile expected a compiler named `riscv-none-elf-gcc`, but the pinned toolchain provided `riscv32-none-elf-gcc`. Fixed by auto-detecting whichever RISC-V compiler is on the path.

3 — Float-ABI mismatch

The standard library in the toolchain was compiled double-float, but the CPU has no FPU and the firmware is soft-float; they refuse to link. Resolved with a freestanding build that supplies its own soft-float 64-bit helpers instead of the incompatible library.

4 — Stack past the end of memory (silent trap)

The firmware was linked assuming 16 KB of data RAM, but the SoC provides 8 KB. The startup code placed the stack pointer 8 KB beyond real memory, so the very first stack write trapped — before the program could print anything. The symptom was a boot with no output. Fixed by matching the firmware’s memory sizes to the hardware.

5 — Flaky debugger and shifting serial ports

The board’s USB debugger periodically re-enumerated, swapping the serial device names and occasionally wedging so that programming itself failed. Handled by addressing the UART through its stable by-identifier path and power-cycling to recover the debugger.

6 — The bus handshake (the key hardware bug)

NEORV32’s external bus drives its strobe as a *single-cycle pulse* and only samples the acknowledge in the cycle *after* the strobe. The first version of the Wishbone adapter asserted its acknowledge *during* the strobe — the one cycle the CPU was not looking. The CPU never saw it, waited out its bus timeout, and raised a store-access fault at the accelerator’s address. Fixed with a handshake state machine that latches the strobe and acknowledges in the following cycle; then re-verified in simulation against a bus master that reproduces NEORV32’s exact timing.

7 — Fitting the FPGA

With the accelerator added, place-and-route reported the design at the device’s utilisation limit and could not find a legal placement. The CPU’s hardware multiply/divide unit was the largest removable block and is unused by the `rv32i` firmware, so dropping it freed enough logic to fit — with the firmware providing software multiply/divide.

8 — Protocol ordering (the last bug)

The driver wrote the `START` command *after* streaming the payload for the memory-mapped data plane, but the accelerator is a start-first stream engine: it only accepts data after being started. Streaming first hit the idle engine, stalled the streaming write, and timed out — again a fault at the data-in register. Fixed by writing `START` *before* streaming, matching the engine and the simulation.

6.1 The pattern that mattered

Bugs 6 and 8 were the two standing between “the CPU cannot reach the accelerator” and the clean result. Both were found the same way: the processor panicked with a precise fault address (`0x9000 0000` for a control write, `0x9000 0044` for a data-in write), and that address named exactly which register — and therefore which mechanism — had failed. Instrumenting the firmware with breadcrumb prints, and a minimal “hello” program to isolate the runtime from the accelerator, turned each silent hang into a located fault.

7 Design Choices and Their Consequences

Choice	Consequence
Golden-model-first generation	RTL and reference share one definition; the verification oracle is free. Bugs were interface bugs, never crypto bugs.
1 round / cycle permutation	Smallest core; fit the small FPGA. Crypto latency is a handful of cycles — not the bottleneck.
Memory-mapped word data plane	Simple, no DMA. But it makes CPU data-movement the dominant cost for large payloads — the reason speed-up falls with size.
XBUS peripheral, not an instruction	Loose coupling, no pipeline changes — but every access is a bus transaction with the CPU in the loop.
Drop the M extension	Freed the logic to fit. Safe because the firmware is <code>rv32i</code> and the bootloader needs no multiply.
Fully-open Nix toolchain	Reproducible and vendor-free — at the cost of the bleeding-edge sharp edges that became challenges 1–3.

8 Limitations and Future Work

- **Payload bound.** 32 bytes of associated data and 32 of message per call. Rate-block streaming for arbitrary lengths is the clear next feature and is a natural extension of the

generator.

- **Data-movement bottleneck.** A DMA path or a wider bus datapath would raise the large-payload numbers, which are currently limited by the word-at-a-time interface, not by the crypto.
- **Mode coverage.** Only AEAD128 runs on hardware; hash/XOF modes exist in the model and could be integrated.
- **Fair end-to-end metric.** Adding a second measurement that wraps the CPU cycle counter around the entire driver call would report the full wall-clock speed-up, not just the accelerator's busy time.
- **Security characterisation.** No constant-time or side-channel analysis has been done; this is a functional and performance result.

9 Conclusion

The project achieved its objectives: a standard-conformant Ascon-AEAD128 accelerator, generated from a verified model, usable from C on a RISC-V SoC, built entirely with open tools, and *validated on silicon* with every test case correct and a measured $18\text{--}57\times$ speed-up over reference software on the same core. Equally valuable is the honest engineering record — eight distinct bugs, each diagnosed from the board's own evidence, spanning the build environment, the toolchain, memory layout, the bus handshake, device fit, and the driver protocol. The two that mattered most, the pulsed-strobe bus handshake and the start-first protocol order, are precisely the kind of hardware/software boundary issue that only a real, end-to-end validation exposes.